INSTITUTO DE ESTUDIOS UNIVERSITARIOS

Ingeniería en Sistemas y Tecnologías de la Información

ACTIVIDAD COLABORATIVA · GESTOR DE PRODUCTOS

Manual paso a paso

Firebase Authentication, Cloud Firestore y Hosting

Integrantes

Saul Domínguez Torres

Jorge Diego López Atenco

Materia: Desarrollo de Aplicaciones Web

Docente: Emmanuel Hernández Ortiz

Fecha: 27 de octubre de 2026

Aplicación publicada: https://k028-e7f83.web.app

Introducción

En esta actividad ampliamos la aplicación de autenticación que vimos en clase (FireApp) para convertirla en un gestor de productos por usuario. La aplicación usa tres servicios de Firebase: **Authentication** para saber quién entró, **Cloud Firestore** para guardar sus productos y **Firebase Hosting** para publicarla en internet.

Este manual explica, paso por paso, cómo quedó construida y cómo se puede reproducir. Cada paso tiene una explicación breve, capturas del código real del proyecto y, donde corresponde, capturas de las consolas de Firebase.

Dato	Valor
Aplicación publicada	https://k028-e7f83.web.app
Repositorio	https://github.com/Saul-MX/FireAppAuth
Proyecto de Firebase	k028-e7f83
Tecnologías	HTML, JavaScript modular (ES modules), Bootstrap 5.3.8 y SDK de Firebase 13.0.0 cargado por CDN
Acceso	Correo y contraseña, Google, Facebook, GitHub y X

Estructura del proyecto

Cada funcionalidad vive en su propio archivo dentro de la carpeta `app/`, y `main.js` los carga todos:

Archivo	Qué hace
`main.js`	Punto de entrada: importa cada módulo y vigila la sesión con `onAuthStateChanged`.
`app/firebase.js`	Configuración e inicialización de Firebase (Authentication y Firestore).
`app/signupForm.js`, `signinForm.js`	Registro e inicio de sesión con correo y contraseña.
`app/googleLogin.js`, `facebookLogin.js`, `githubLogin.js`, `twitterLogin.js`	Inicio de sesión con cada proveedor OAuth.
`app/logout.js`	Cierre de sesión.
`app/loginCheck.js`	Oculta Signin/Signup y muestra el módulo de productos (y al revés).

Archivo	Qué hace
`app/authErrors.js` , `firestoreErrors.js`	Mensajes de error claros para Authentication y Firestore.
`app/imagePicker.js`	Selección, validación (1 a 5) y vista previa de las imágenes.
`app/imageProcessor.js`	Redimensiona y comprime cada imagen con canvas y la convierte a Base64.
`app/productForm.js`	Formulario de registro y de edición.
`app/productsDb.js`	Todas las operaciones con Firestore (crear, leer, editar y eliminar).
`app/productList.js`	Lista en tiempo real con imágenes; la limpia al cerrar sesión.
`app/productDelete.js`	Confirmación y eliminación de un producto.
`app/toast.js`	Avisos breves en pantalla.

Desarrollo

Paso 1. Activación de Firestore y estructura de datos

Objetivo: crear la base de datos y definir dónde se guarda cada dato

La base de datos se crea en la consola de Firebase, en **Build → Firestore Database → Create database**. Se elige la ubicación de los datos y, en el paso de reglas, el **modo de prueba** (*Start in test mode*), que permite leer y escribir mientras se desarrolla. Al terminar, Firestore queda listo para recibir datos.

Las reglas del modo de prueba dejan el acceso abierto durante un tiempo limitado. Es suficiente para esta actividad, pero no es seguro para producción: cualquiera que conozca el proyecto podría leer y modificar los datos. En la investigación (documento C) se explica qué cambiaría con reglas de seguridad; aquí solo se mencionan y no se implementan.

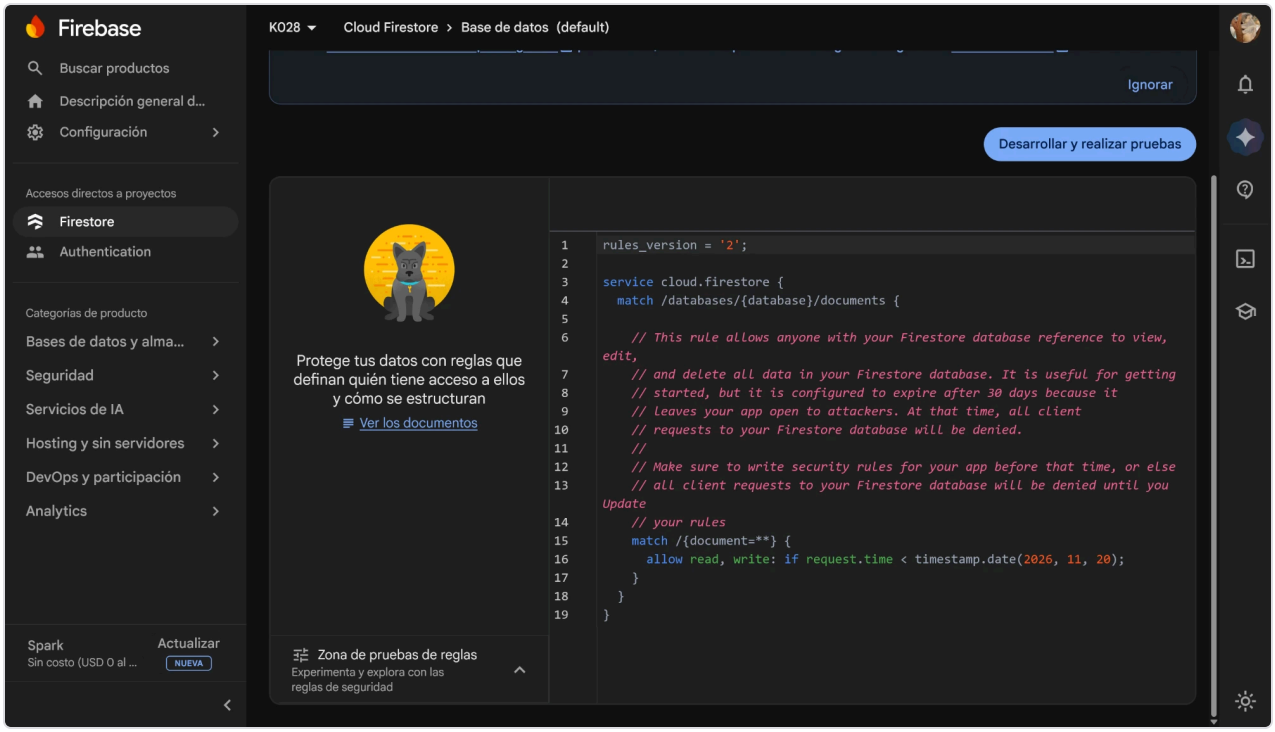


Figura 1. Reglas del modo de prueba en la consola.

Estructura de datos

Cada usuario guarda sus productos dentro de su propia ruta, construida con el **uid** que le asigna Authentication. Las imágenes van en una subcolección del producto, con un documento por imagen:

```
users/{uid}/products/{productId}
    nombre, descripcion, precio, categoria, existencias,
    totalImagenes, fechaCreacion, fechaActualizacion

users/{uid}/products/{productId}/images/{imageId}
    data (Base64), nombreArchivo, orden
```

Campo	Tipo	Descripción
`nombre`	string	Nombre del producto (máximo 80 caracteres).
`descripcion`	string	Descripción (máximo 500 caracteres).
`precio`	number	Precio en pesos, redondeado a centavos.
`categoria`	string	Una de las categorías del formulario.
`existencias`	number	Número entero igual o mayor a 0.
`totalImagenes`	number	Cantidad de imágenes (de 1 a 5); evita leer la subcolección solo para contarlas.
`fechaCreacion` , `fechaActualizacion`	timestamp	Se escriben con `serverTimestamp()` .
`data` (imagen)	string	Data URL en Base64 de la imagen ya comprimida (máximo 300 KB).
`nombreArchivo` (imagen)	string	Nombre original del archivo.
`orden` (imagen)	number	Posición de la imagen; 0 es la portada.

¿Por qué una subcolección para las imágenes? Firestore limita cada documento a 1 MiB. Si las cinco imágenes fueran un campo del producto, un solo documento podría acercarse a ese límite y, además, escuchar la lista de productos descargaría todas las imágenes. Con una subcolección cada imagen es un documento independiente (de unos 300 KB como máximo) y la lista de productos se mantiene ligera.

app/firebase.js líneas 1–22

```
// Configuración e inicialización de Firebase (SDK modular v13 desde CDN)
import { initializeApp } from "https://www.gstatic.com/firebasejs/13.0.0/firebase-app.js";
import { getAuth } from "https://www.gstatic.com/firebasejs/13.0.0/firebase-auth.js";
import { getFirestore } from "https://www.gstatic.com/firebasejs/13.0.0/firebase-firestore.js";

// Datos del proyecto obtenidos en la consola de Firebase.
// Son públicos: identifican al proyecto, no dan acceso por sí solos.
// La protección real está en los dominios autorizados y las reglas de seguridad.
const firebaseConfig = {
    apiKey: "AIzaSyAcMzFHMrYSmZ09eoVOMNTJ1GTt1CAhBmM",
    authDomain: "k028-e7f83.firebaseapp.com",
    projectId: "k028-e7f83",
    storageBucket: "k028-e7f83.firebasestorage.app",
    messagingSenderId: "705611004597",
    appId: "1:705611004597:web:f86d2f6eab601b70f35cfc"
};

// Inicializa la app y exporta las instancias de autenticación y de Cloud Firestore
// para que el resto de los módulos las reutilicen
export const app = initializeApp(firebaseConfig);
export const auth = getAuth(app);
export const db = getFirestore(app);
```

Figura 2. Inicialización de Firebase: Authentication y Firestore (getFirestore).

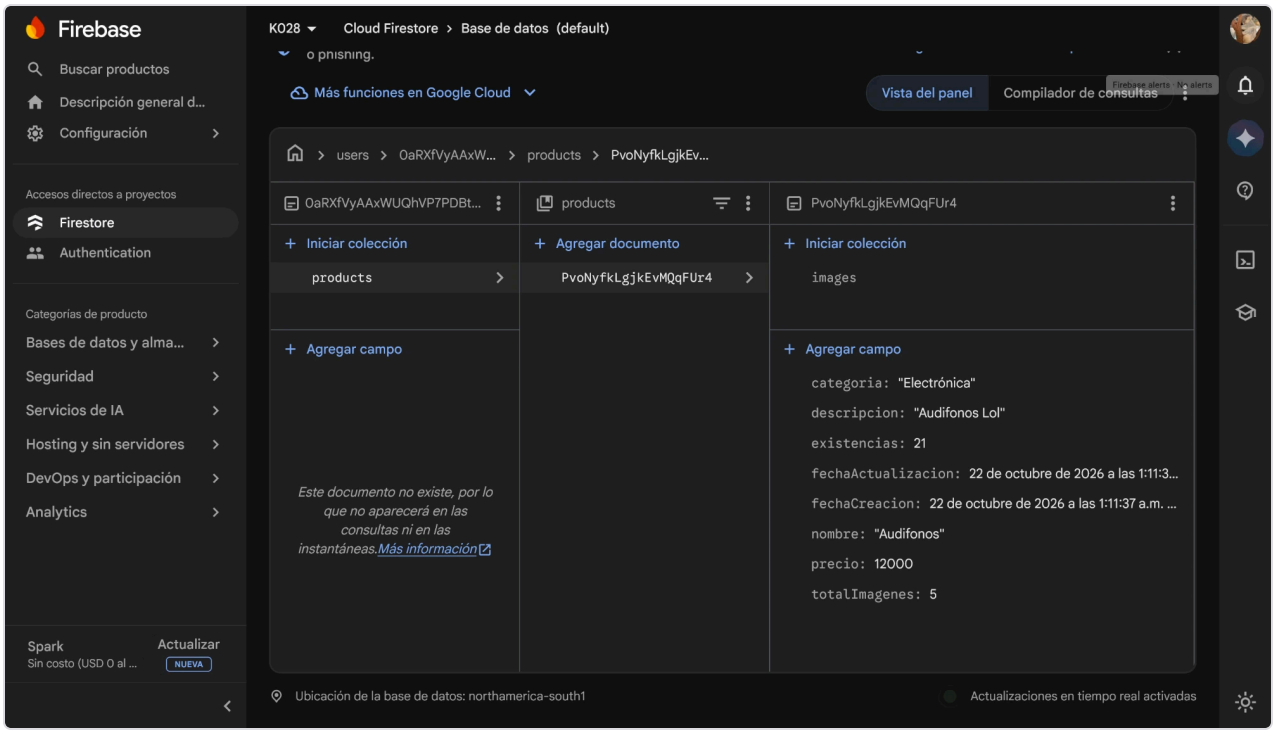


Figura 3. Datos guardados: producto y subcolección images.

Paso 2. Formulario y validación de imágenes

Objetivo: capturar los datos del producto y validar de 1 a 5 imágenes antes de guardar

El formulario es una ventana modal de Bootstrap con nombre, descripción, precio, categoría y existencias. Todos son obligatorios y llevan restricciones de HTML (`required`, `min`, `maxlength`). Las imágenes se eligen con un `input type="file"` con `multiple` y `accept="image/*"`, dentro de una zona donde también se pueden arrastrar archivos.

index.html líneas 340–356

```
<form id="product-form">
    <div class="product-fields">
        <div class="form-field field-wide">
            <label for="product-name">Nombre</label>
            <input type="text" id="product-name" placeholder="Ej. Audífonos inalámbricos"
                maxlength="80" required class="form-control">
        </div>
        <div class="form-field field-wide">
            <label for="product-description">Descripción</label>
            <textarea id="product-description" rows="3" placeholder="Describe brevemente el producto"
                maxlength="500" required class="form-control"></textarea>
        </div>
        <div class="form-field">
            <label for="product-price">Precio (MXN)</label>
            <input type="number" id="product-price" placeholder="0.00" min="0" step="0.01"
                inputmode="decimal" required class="form-control">
        </div>
```

Figura 4. Campos del formulario de productos.

index.html líneas 384–397

```
<label class="image-dropzone" id="image-dropzone">
    <input type="file" id="product-images" accept="image/*" multiple class="visually-hidden">
    <span class="image-dropzone-icon" aria-hidden="true">
        <svg viewBox="0 0 24 24" fill="none">
            <rect x="3.5" y="4.5" width="17" height="15" rx="3" stroke="currentColor" stroke-
              width="1.6" />
            <circle cx="9" cy="10" r="1.8" stroke="currentColor" stroke-width="1.6" />
            <path d="m4 17 4.6-4.4a1.6 1.6 0 0 1 2.2 0l4.2 4m-1.6-1.6 1.9-1.8a1.6 1.6 0 0 1 2.2 0l2.5
              2.4" stroke="currentColor" stroke-width="1.6" stroke-linecap="round" stroke-
              linejoin="round" />
        </svg>
    </span>
    <strong id="image-dropzone-text">Haz clic o arrastra tus imágenes aquí</strong>
    <small>De 1 a 5 imágenes JPG, PNG o WebP de hasta 10 MB cada una</small>
</label>
<div class="form-alert" id="image-feedback" role="alert" hidden></div>
<ul class="image-previews" id="image-previews" aria-label="Vista previa de las imágenes"></ul>
```

Figura 5. Zona de selección de imágenes, mensaje de error y vista previa.

Reglas de validación

Regla	Valor	Mensaje cuando no se cumple
Cantidad	De 1 a 5 imágenes	«Agrega al menos 1 imagen del producto.» / «Puedes subir máximo 5 imágenes por producto…»
Tipo de archivo	Solo `image/*`	«Solo se permiten imágenes: …no es una imagen.»
Peso del original	Máximo 10 MB cada una	«Cada imagen debe pesar máximo 10 MB…»
Peso guardado	Máximo 300 KB (Base64)	No hay mensaje: la imagen se reduce automáticamente (paso 3).

La validación ocurre en dos momentos. Al **elegir los archivos**, `validateFiles` revisa el tipo, el peso y que, sumando las que ya estaban, no se pase de 5; si un archivo no cumple, no se agrega ninguno y se explica el motivo. Al **guardar**, `validateImageCount` comprueba que haya entre 1 y 5, así que subir 0 o más de 5 siempre muestra un mensaje claro. Cada imagen aceptada aparece con su vista previa, su peso antes y después de comprimirla y un botón para quitarla; la primera es la portada.

app/imagePicker.js — líneas 91–118

```
91  // Revisa los archivos elegidos antes de procesarlos: tipo, peso y cantidad.
92  // Devuelve el mensaje de error, o una cadena vacía si todo está bien.
93  const validateFiles = (files) => {
94      const notImages = files.filter((file) => !file.type.startsWith('image/'));
95      if (notImages.length) {
96          const verb = notImages.length === 1 ? 'no es una imagen' : 'no son imágenes';
97          return `Solo se permiten imágenes: ${fileNames(notImages)} ${verb}.`;
98      }
99
100     const tooLarge = files.filter((file) => file.size > MAX_FILE_SIZE);
101     if (tooLarge.length) {
102         const verb = tooLarge.length === 1 ? 'pesa' : 'pesan';
103         return `Cada imagen debe pesar máximo ${MAX_FILE_MB} MB: ${fileNames(tooLarge)} ${verb} más.`;
104     }
105
106     if (images.length + files.length > MAX_IMAGES) {
107         const available = MAX_IMAGES - images.length;
108         if (images.length === 0) {
109             return `Puedes subir máximo ${MAX_IMAGES} imágenes por producto y seleccionaste
                ${files.length}.`;
110         }
111         if (available === 0) {
112             return `El producto ya tiene ${MAX_IMAGES} imágenes, el máximo permitido. Quita alguna para
                agregar otra.`;
113         }
114         return `Puedes subir máximo ${MAX_IMAGES} imágenes por producto. Ya tienes ${images.length},
              solo puedes agregar ${available} más.`;
115     }
116
117     return '';
118 };
```

Figura 6. validateFiles: tipo, peso y cantidad al elegir los archivos.

app/imagePicker.js líneas 204–214

```
204 // Valida la cantidad antes de guardar (mínimo 1, máximo 5) y muestra el error si no se cumple
205 export const validateImageCount = () => {
206     let error = '';
207     if (images.length < MIN_IMAGES) {
208         error = `Agrega al menos ${MIN_IMAGES} imagen del producto.`;
209     } else if (images.length > MAX_IMAGES) {
210         error = `Puedes subir máximo ${MAX_IMAGES} imágenes por producto.`;
211     }
212     showError(error);
213     return !error;
214 };
```

Figura 7. validateImageCount: mínimo 1 y máximo 5 antes de guardar.

CAPTURA PENDIENTE

La aplicación mostrando el mensaje de error al seleccionar más de 5 imágenes (por ejemplo, 6).

capturas/app-validacion-max.png

Figura 8. Error al pasarse del máximo de 5 imágenes.

CAPTURA PENDIENTE

El formulario con varias imágenes cargadas: contador (por ejemplo 3 / 5), vista previa, peso y etiqueta «Portada».

capturas/app-vista-previa.png

Figura 9. Vista previa de las imágenes antes de guardar.

> **Importante.** Esta validación sirve para dar respuesta inmediata al usuario, pero no sustituye a una validación en el servidor: un usuario con conocimientos podría saltarse el navegador. Se explica en el documento C.

Paso 3. Procesamiento y guardado de imágenes

Objetivo: reducir cada imagen en el navegador y guardarla como Base64 en Firestore

Antes de guardarse, cada imagen pasa por `imageProcessor.js`, que trabaja en tres etapas:

1. **Lectura.** `FileReader` lee el archivo como Data URL y se carga en un `<img>`.
2. **Redimensionado.** Se dibuja en un `<canvas>` con el lado más largo de 800 px como máximo (nunca se agranda).
3. **Compresión.** `canvas.toDataURL()` la exporta en WebP (o JPEG si el navegador no lo soporta) probando calidades de 0.8 a 0.4 hasta que el texto Base64 mida 300 KB o menos; si ni así cabe, reduce las dimensiones 20 % y repite.

app/imageProcessor.js — líneas 1–16

```
// Procesa cada imagen en el navegador antes de guardarla en Firestore:
// 1) la lee con FileReader, 2) la redimensiona en un <canvas> y
// 3) la comprime y la convierte a Base64 (Data URL).
// Firestore limita cada documento a 1 MiB, por eso cada imagen se deja en máximo 300 KB.

const MAX_SIDE = 800;                        // px máximos del lado más largo
const MAX_IMAGE_BYTES = 300 * 1024;          // tamaño máximo del texto Base64 que se guarda
const QUALITIES = [0.8, 0.7, 0.6, 0.5, 0.4];   // calidades a probar, de mejor a menor

// WebP pesa menos que JPEG con la misma calidad, pero algunos navegadores (como Safari)
// no lo generan desde canvas y devuelven PNG; en ese caso se usa JPEG.
const OUTPUT_TYPE = (() => {
    const canvas = document.createElement('canvas');
    canvas.width = canvas.height = 1;
    return canvas.toDataURL('image/webp').startsWith('data:image/webp') ? 'image/webp' : 'image/jpeg';
})();
```

Figura 10. Límites y formato de salida (WebP con respaldo a JPEG).

app/imageProcessor.js líneas 34–68

```
// Redimensiona y comprime una imagen; devuelve los datos listos para Firestore
export const processImage = async (file) => {
    const image = await loadImage(await readFile(file));
    const { naturalWidth: width, naturalHeight: height } = image;
    if (!width || !height) throw new Error('La imagen no tiene medidas válidas');

    const canvas = document.createElement('canvas');
    const context = canvas.getContext('2d');
    // Escala para que el lado más largo mida como máximo MAX_SIDE (nunca se agranda)
    let scale = Math.min(1, MAX_SIDE / Math.max(width, height));

    while (true) {
        canvas.width = Math.max(1, Math.round(width * scale));
        canvas.height = Math.max(1, Math.round(height * scale));
        // Fondo blanco: JPEG no tiene transparencia y esas zonas saldrían negras
        context.fillStyle = '#fff';
        context.fillRect(0, 0, canvas.width, canvas.height);
        context.drawImage(image, 0, 0, canvas.width, canvas.height);

        // Baja la calidad poco a poco hasta que el resultado quepa en el límite
        for (const quality of QUALITIES) {
            const data = canvas.toDataURL(OUTPUT_TYPE, quality);
            if (data.length <= MAX_IMAGE_BYTES) {
                return {
                    data,                          // Data URL: "..."
                    nombreArchivo: file.name,
                    originalSize: file.size        // bytes del archivo original (solo para mostrarlo)
                };
            }
        }

        // Ni con la calidad más baja cupo: se reduce la medida un 20 % y se intenta de nuevo
        scale *= 0.8;
    }
};
```

Figura 11. processImage: redimensiona y comprime hasta cumplir el límite.

Base64 aumenta el tamaño aproximadamente un tercio (3 bytes se escriben con 4 caracteres). Por eso el límite de 300 KB se mide sobre el texto ya codificado: 300 KB de Base64 equivalen a unos 225 KB de imagen real, muy por debajo del límite de 1 MiB por documento.

Guardado en Firestore

`createProduct` usa un **lote** (`writeBatch`): el producto y cada una de sus imágenes se escriben juntos, así que o se guarda todo o no se guarda nada y nunca queda un producto sin imágenes. `doc()` sin identificador genera uno automático; cada imagen es un documento de la subcolección `images` con su `orden` (0 es la portada).

app/productsDb.js líneas 13–15, 36–57

```
13 // Referencias a las colecciones del usuario
14 const productsCollection = (uid) => collection(db, 'users', uid, 'products');
15 const imagesCollection = (uid, productId) => collection(db, 'users', uid, 'products', productId,
     'images');
   …
36 // Registra un producto con sus imágenes.
37 // Se usa un lote (writeBatch): el producto y todas sus imágenes se escriben juntos,
38 // así que o se guarda todo o no se guarda nada (no quedan productos sin imágenes).
39 export const createProduct = async (uid, product, images) => {
40     const batch = writeBatch(db);
41     // doc() sin id genera un id automático, como addDoc, pero sin escribir todavía
42     const productRef = doc(productsCollection(uid));
43
44     batch.set(productRef, {
45         ...product,
46         totalImagenes: images.length,
47         fechaCreacion: serverTimestamp(),
48         fechaActualizacion: serverTimestamp()
49     });
50
51     // Cada imagen es un documento independiente dentro de la subcolección images
52     images.forEach((image, index) => {
53         batch.set(doc(imagesCollection(uid, productRef.id)), imageFields(image, index));
54     });
55
56     await batch.commit();
57 };
```

Figura 12. createProduct: producto e imágenes en un solo lote.

app/productForm.js líneas 117–147

```
117 productForm.addEventListener('submit', async (e) => {
118     // Evita que el formulario recargue la página
119     e.preventDefault();
120     if (saving) return;
121     showError('');
122 
123     if (isBusy()) {
124         showError('Espera a que terminen de cargarse las imágenes.');
125         return;
126     }
127     // Primero los campos y después las imágenes (de 1 a 5)
128     const product = readProduct();
129     if (!product || !validateImageCount()) return;
130 
131     const session = formSession;
132     const productId = editingId;
133     setSaving(true);
134 
135     try {
136         // Siempre con el uid del usuario autenticado: users/{uid}/products
137         const uid = auth.currentUser.uid;
138         if (productId) {
139             await updateProduct(uid, productId, product, getImages(), getRemovedIds());
140         } else {
141             await createProduct(uid, product, getImages());
142         }
143         if (session !== formSession) return;
144 
145         setSaving(false);
146         modal().hide();
147         showToast(productId ? 'Cambios guardados' : 'Producto registrado');
```

Figura 13. Envío del formulario: valida los campos, valida las imágenes y guarda con el uid del usuario.

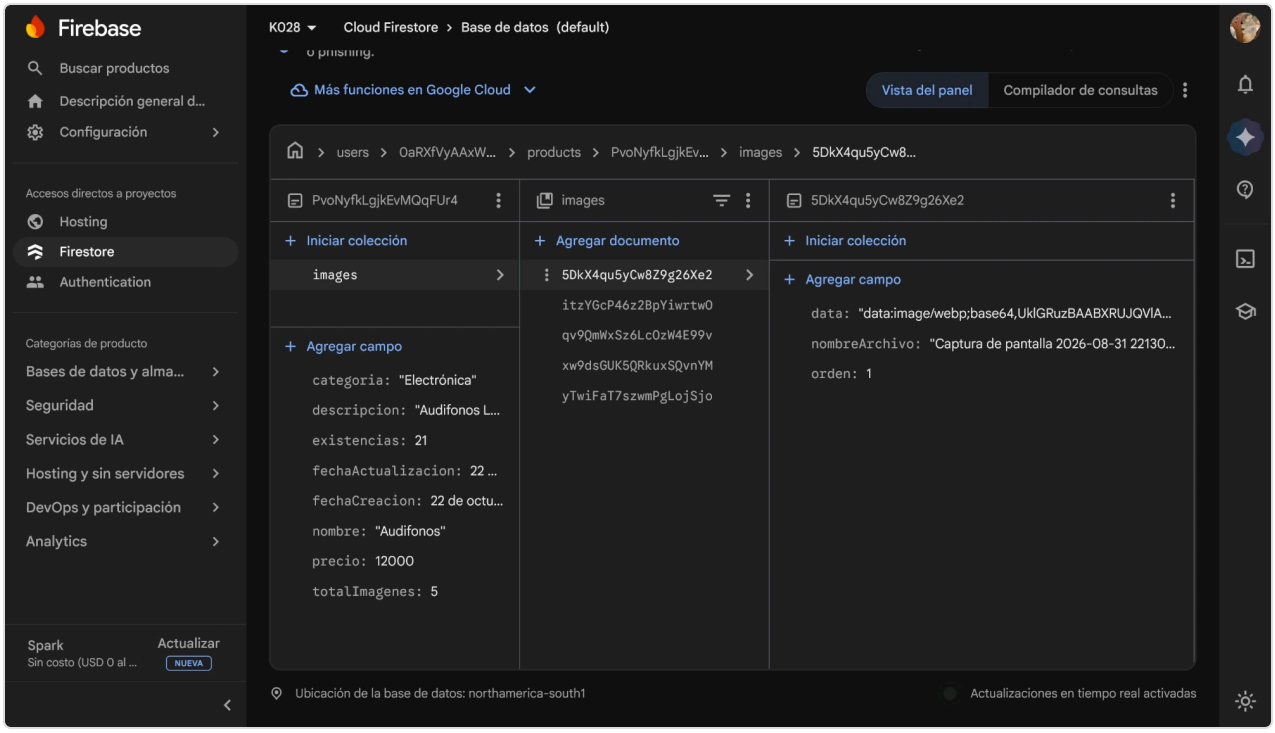


Figura 14. Una imagen guardada como documento de la subcolección images.

Paso 4. Lectura, edición y eliminación filtrando por uid

Objetivo: que cada usuario vea y administre solo sus propios productos

El filtro por usuario se logra desde el código: todas las rutas se construyen con el uid del usuario autenticado (`auth.currentUser.uid`) y la aplicación solo consulta `users/{uid}/products`. Nunca consulta la colección de otro usuario.

Lectura

`listenProducts` usa `onSnapshot` sobre una consulta ordenada con `orderBy`: la lista se actualiza en tiempo real cuando se crea, edita o elimina un producto, sin recargar la página. `getProductImages` usa `getDocs` (lectura única) para traer las imágenes de cada producto, y `productList.js` guarda en caché las que ya leyó, comparando `fechaActualizacion`, para ahorrar lecturas de la cuota gratuita.

app/productsDb.js — líneas 13–15, 24–34

```
13  // Referencias a las colecciones del usuario
14  const productsCollection = (uid) => collection(db, 'users', uid, 'products');
15  const imagesCollection = (uid, productId) => collection(db, 'users', uid, 'products', productId,
      'images');
      …
24  // Escucha en tiempo real (onSnapshot) los productos del usuario, del más nuevo al más antiguo.
25  // Firestore vuelve a llamar a onChange cada vez que se crea, edita o elimina un producto.
26  // Devuelve la función que detiene la escucha (se usa al cerrar sesión).
27  export const listenProducts = (uid, onChange, onError) =>
28      onSnapshot(query(productsCollection(uid), orderBy('fechaCreacion', 'desc')), onChange, onError);
29
30  // Lectura única (getDocs) de las imágenes de un producto, en su orden
31  export const getProductImages = async (uid, productId) => {
32      const snapshot = await getDocs(query(imagesCollection(uid, productId), orderBy('orden')));
33      return snapshot.docs.map((image) => ({ ...image.data(), id: image.id }));
34  };
```

Figura 15. Rutas por uid, escucha en tiempo real y lectura de imágenes.

app/productList.js líneas 174–196

```
174 // Empieza a escuchar users/{uid}/products del usuario que inició sesión
175 export const showProducts = (user) => {
176     clearProducts();
177     currentUid = user.uid;
178     userName.textContent = user.displayName || user.email || 'tu cuenta';
179     userPath.textContent = `users/${user.uid}/products`;
180     productCount.textContent = 'Cargando…';
181     showStatus('Cargando tus productos…');
182 
183     unsubscribe = listenProducts(user.uid, (snapshot) => {
184         products = snapshot.docs.map((doc) => ({
185             // 'estimate' da una fecha aproximada mientras Firestore confirma la escritura
186             ...doc.data({ serverTimestamps: 'estimate' }),
187             id: doc.id,
188             pending: doc.metadata.hasPendingWrites
189         }));
190         renderList();
191         loadImages(user.uid);
192     }, (error) => {
193         console.error(error);
194         showStatus(`No se pudieron cargar tus productos. ${getDbErrorMessage(error)}`);
195     });
196 };
```

Figura 16. showProducts: empieza a escuchar users/{uid}/products del usuario con sesión.

CAPTURA PENDIENTE

La sección «Mis productos» con las tarjetas: imagen principal, miniaturas, categoría, existencias, precio y botones Editar y Eliminar.

capturas/app-lista.png

Figura 17. Listado de productos con sus imágenes.

Edición

Al pulsar **Editar**, el formulario se abre con los datos del producto y sus imágenes guardadas (leídas con `getDocs`). `updateProduct` actualiza el producto y sincroniza las imágenes en un solo lote: borra las que se quitaron, reordena las que se conservaron y agrega las nuevas.

app/productsDb.js líneas 59–84

```
// Actualiza un producto y sincroniza sus imágenes en un solo lote:
// borra las que el usuario quitó, reordena las que conservó y agrega las nuevas.
export const updateProduct = async (uid, productId, product, images, removedIds) => {
    const batch = writeBatch(db);

    batch.update(doc(productsCollection(uid), productId), {
        ...product,
        totalImagenes: images.length,
        fechaActualizacion: serverTimestamp()
    });

    removedIds.forEach((imageId) => {
        batch.delete(doc(imagesCollection(uid, productId), imageId));
    });

    images.forEach((image, index) => {
        if (image.id) {
            // Ya estaba guardada: solo cambia su posición
            batch.update(doc(imagesCollection(uid, productId), image.id), { orden: index });
        } else {
            batch.set(doc(imagesCollection(uid, productId)), imageFields(image, index));
        }
    });

    await batch.commit();
};
```

Figura 18. updateProduct: actualiza el producto y sincroniza sus imágenes.

Eliminación

Firestore **no borra las subcolecciones** al eliminar un documento. Por eso `deleteProduct` primero lee las imágenes y las elimina en el mismo lote que el producto; así no quedan imágenes huérfanas. Antes se pide confirmación al usuario.

app/productsDb.js líneas 86–97

```
// Elimina un producto junto con todas sus imágenes.
// Firestore NO borra las subcolecciones al borrar un documento, por eso primero
// se leen las imágenes y se eliminan en el mismo lote que el producto.
export const deleteProduct = async (uid, productId) => {
    const images = await getDocs(imagesCollection(uid, productId));
    const batch = writeBatch(db);

    images.forEach((image) => batch.delete(image.ref));
    batch.delete(doc(productsCollection(uid), productId));

    await batch.commit();
};
```

Figura 19. deleteProduct: elimina las imágenes y el producto.

app/productDelete.js líneas 32–48

```
confirmButton.addEventListener('click', async () => {
    if (!selected) return;
    const product = selected;
    confirmButton.disabled = true;
    confirmButton.textContent = 'Eliminando…';

    try {
        // Borra el documento del producto y su subcolección de imágenes
        await deleteProduct(auth.currentUser.uid, product.id);
        showToast(`Se eliminó «${product.nombre}»`);
    } catch (error) {
        console.error(error);
        showToast(getDbErrorMessage(error), 'error');
    } finally {
        if (selected === product) closeDeleteDialog();
    }
});
```

Figura 20. Confirmación de eliminación usando el uid del usuario autenticado.

Cierre de sesión: no quedan productos del usuario anterior

`main.js` vigila la sesión con `onAuthStateChanged`. Con sesión llama a `showProducts(user)`; sin sesión llama a `clearProducts()`, que detiene la escucha de Firestore, vacía la lista y la caché, y cierra el formulario y los diálogos. `loginCheck.js` oculta Signin/Signup y muestra el módulo de productos al entrar, y hace lo contrario al salir.

main.js líneas 19–30

```
// Se ejecuta al cargar la página y cada vez que se inicia o cierra sesión.
// user es el usuario autenticado o null si no hay sesión.
onAuthStateChanged(auth, (user) => {
    loginChecker(user);

    // Con sesión se escuchan solo los productos de su uid; sin sesión se limpia la pantalla
    if (user) {
        showProducts(user);
    } else {
        clearProducts();
    }
});
```

Figura 21. onAuthStateChanged: muestra los productos del usuario o limpia la pantalla.

app/productList.js líneas 198–214

```
198 // Deja de escuchar y borra de la pantalla todo lo del usuario anterior
199 export const clearProducts = () => {
200     unsubscribe?.();
201     unsubscribe = null;
202     currentUid = null;
203     products = [];
204     cards.clear();
205     imageCache.clear();
206     list.replaceChildren();
207     productCount.textContent = '';
208     userName.textContent = '';
209     userPath.textContent = '';
210     emptyState.hidden = true;
211     showStatus('');
212     closeProductForm();
213     closeDeleteDialog();
214 };
```

Figura 22. clearProducts: detiene la escucha y borra todo lo del usuario anterior.

app/loginCheck.js líneas 6–16

```
6  export const loginChecker = user => {
7      if (user) {
8          // Sesión activa: se muestran Logout y los productos; se ocultan Signin/Signup y la bienvenida
9          loggedInLinks.forEach(link => link.style.display = 'block')
10         loggedOutLinks.forEach(link => link.style.display = 'none')
11     } else {
12         // Sin sesión: se muestran Signin/Signup y la bienvenida; se ocultan Logout y los productos
13         loggedInLinks.forEach(link => link.style.display = 'none')
14         loggedOutLinks.forEach(link => link.style.display = 'block')
15     }
16 }
```

Figura 23. Muestra u oculta las secciones según haya sesión.

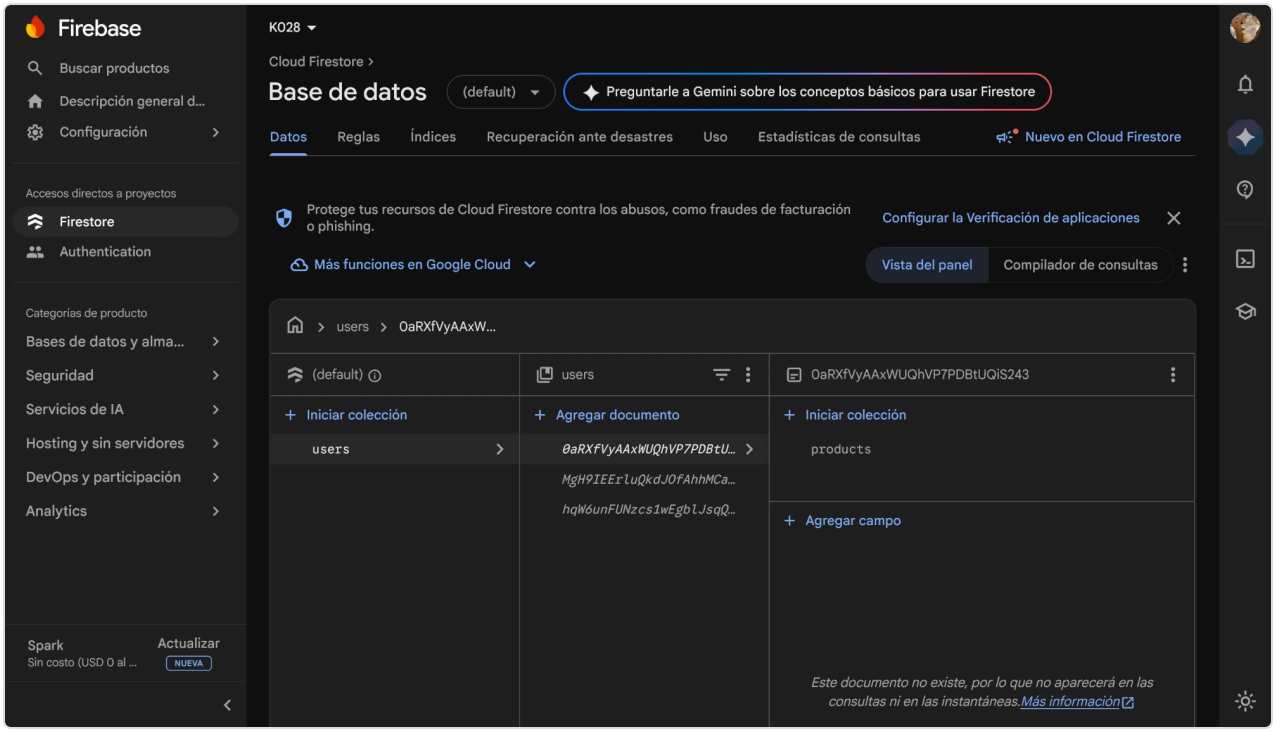


Figura 24. Cada usuario tiene sus productos bajo su propio uid.

Paso 5. Firebase CLI, firebase init hosting y firebase deploy

Objetivo: publicar la aplicación en https://k028-e7f83.web.app

Firebase Hosting sirve los archivos del proyecto desde internet con HTTPS. Se publica desde la terminal con Firebase CLI:

1. **Instalar Node.js** (incluye `npm`) y comprobarlo con `node --version`.
2. **Instalar Firebase CLI:** `npm install -g firebase-tools` y comprobar con `firebase --version`.
3. **Iniciar sesión:** `firebase login` abre el navegador para autenticarse con la cuenta de Google dueña del proyecto.
4. **Inicializar Hosting** desde la carpeta del proyecto: `firebase init hosting`.
5. **Publicar:** `firebase deploy --only hosting`.

```
$ npm install -g firebase-tools
$ firebase --version
$ firebase login
$ firebase init hosting
$ firebase deploy --only hosting

=== Deploying to 'k028-e7f83'...
✔  hosting[k028-e7f83]: release complete
✔  Deploy complete!
Hosting URL: https://k028-e7f83.web.app
```

Figura 25. Comandos de Firebase CLI y resultado esperado del deploy.



Figura 26. Instalación de Firebase CLI e inicio de sesión.

Al ejecutar `firebase init hosting` la CLI hace estas preguntas; en nuestro caso las respuestas fueron:

Pregunta de la CLI	Respuesta
Please select an option	Use an existing project → `k028-e7f83`
What do you want to use as your public directory?	`.` (el `index.html` está en la raíz del proyecto; la CLI propone `public` por defecto)
Configure as a single-page app?	No
Set up automatic builds and deploys with GitHub?	No

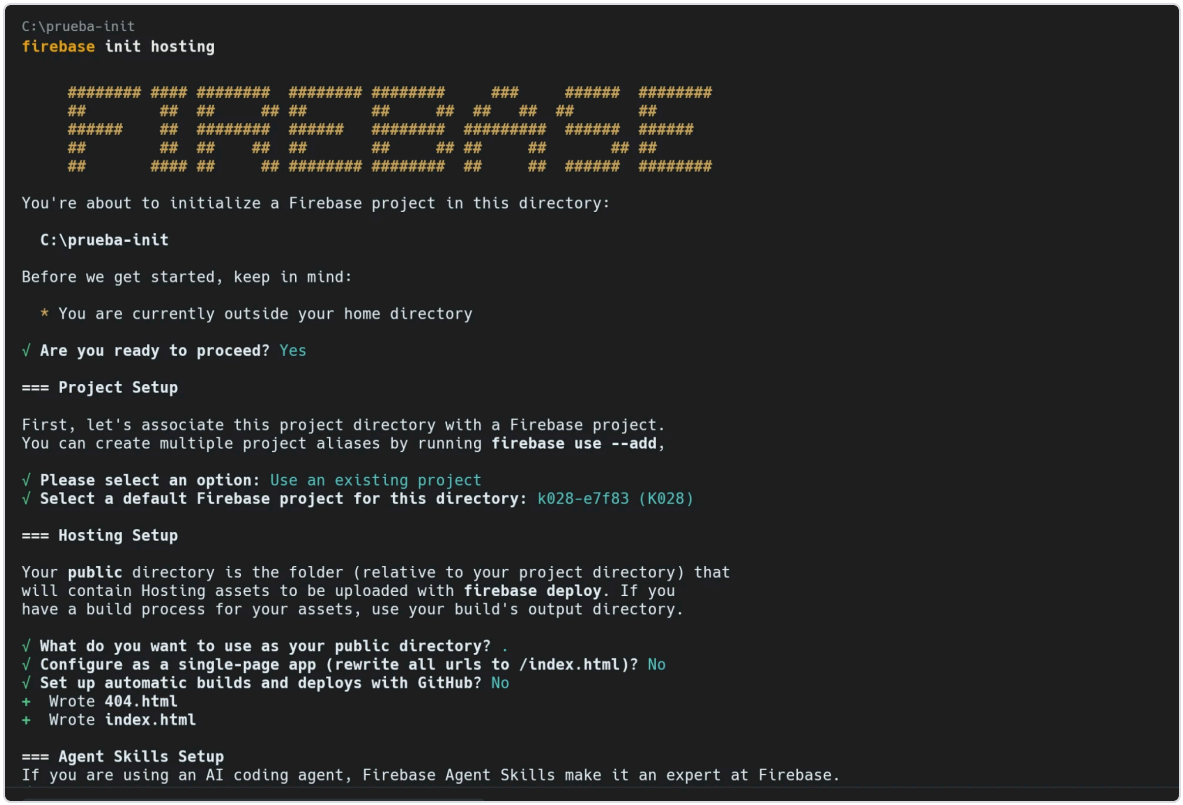


Figura 27. Inicialización de Hosting con firebase init hosting.

La CLI genera dos archivos: `firebase.json`, con la configuración de Hosting, y `.firebaserc`, que guarda el proyecto por defecto. En `firebase.json`, `public` es la carpeta que se publica (aquí la raíz), `ignore` evita subir archivos que no deben ser públicos (`firebase.json`, archivos ocultos, `node_modules`, registros y los PDF de la carpeta `entregables`) y `headers` con `Cache-Control: no-cache` hace que el navegador revise siempre si hay una versión nueva.

firebase.json líneas 1–24

```
{
  "hosting": {
    "public": ".",
    "ignore": [
      "firebase.json",
      "**/.*",
      "**/.*/**",
      "**/node_modules/**",
      "**/*.log",
      "entregables/**"
    ],
    "headers": [
      {
        "source": "**",
        "headers": [
          {
            "key": "Cache-Control",
            "value": "no-cache"
          }
        ]
      }
    ]
  }
}
```

Figura 28. Configuración de Firebase Hosting.

.firebaserc líneas 1–5

```
{
  "projects": {
    "default": "k028-e7f83"
  }
}
```

Figura 29. Proyecto de Firebase asociado a la carpeta.

CAPTURA PENDIENTE

La terminal al terminar firebase deploy --only hosting, con «Deploy complete!» y la Hosting URL.

capturas/cli-deploy.png

Figura 30. Despliegue terminado con la URL pública.

Verificación en la consola y versiones

En **Build → Hosting** aparece el dominio publicado y el *Release history*. Cada `deploy` crea una versión nueva; si algo sale mal, la opción **Roll back** vuelve a servir una versión anterior.

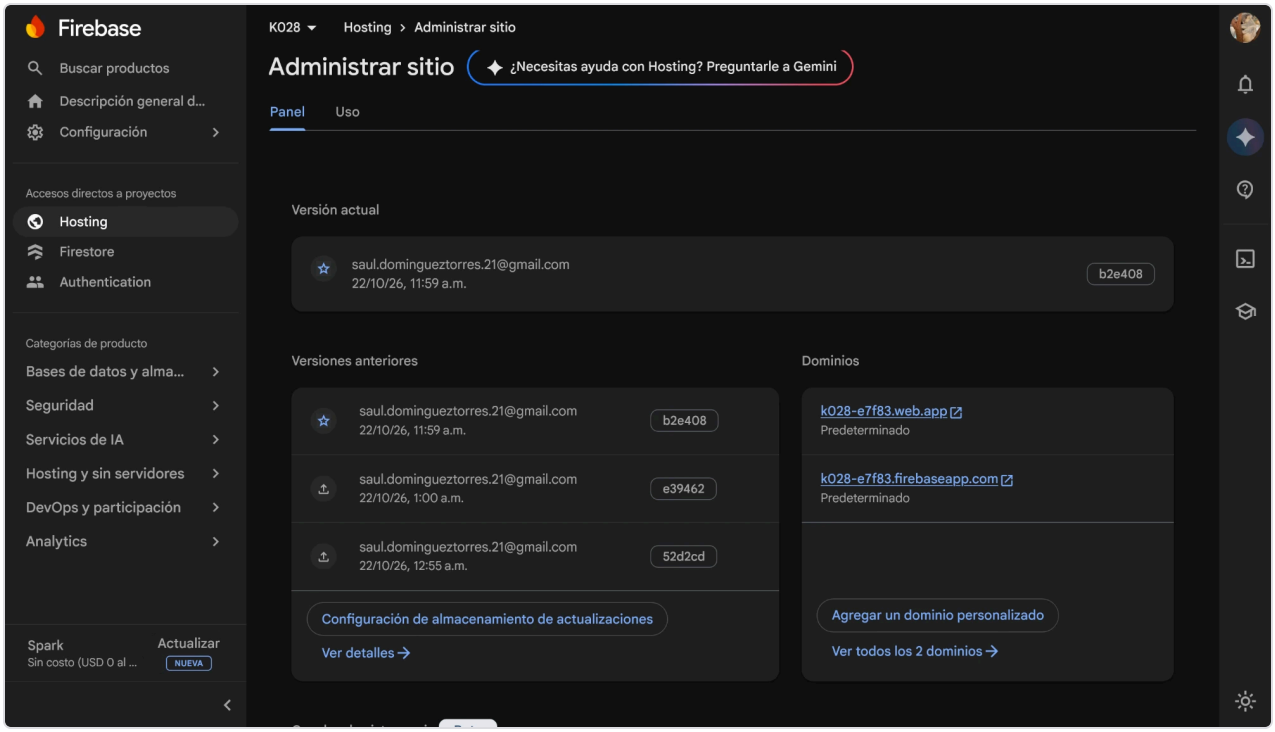


Figura 31. Panel de Hosting con la versión publicada.

CAPTURA PENDIENTE

El navegador abriendo la URL pública https://k028-e7f83.web.app con la aplicación funcionando (se ve el candado de HTTPS).

`capturas/app-publicada.png`

Figura 32. La aplicación publicada en Internet.

Paso 6. Verificación de dominios autorizados

Objetivo: comprobar que el inicio de sesión funciona desde el sitio publicado

Google, Facebook, GitHub y X abren una ventana emergente (`signInWithPopup`). Firebase solo permite hacerlo desde los **dominios autorizados** del proyecto: si el dominio desde el que se abre la aplicación no está en la lista, el inicio de sesión falla con el error `auth/unauthorized-domain`. Por eso, después de publicar, hay que revisarlos:

1. Abrir **Authentication → Settings → Authorized domains** en la consola de Firebase.
2. Comprobar que estén **k028-e7f83.web.app** y **k028-e7f83.firebaseapp.com**. Normalmente ya vienen incluidos; si falta alguno, usar **Add domain**.
3. `localhost` solo se necesita para probar en la computadora; en proyectos creados después del 28 de abril de 2025 no se agrega por defecto.
4. Abrir **Authentication → Sign-in method** y confirmar que estén habilitados Correo/Contraseña, Google, Facebook, GitHub y X.
5. En las consolas de Facebook, GitHub y X debe estar registrada la URI de redirección que muestra Firebase: `https://k028-e7f83.firebaseapp.com/__/auth/handler` (es el `authDomain` de `app/firebase.js`).
6. Probar desde la URL pública al menos dos proveedores.

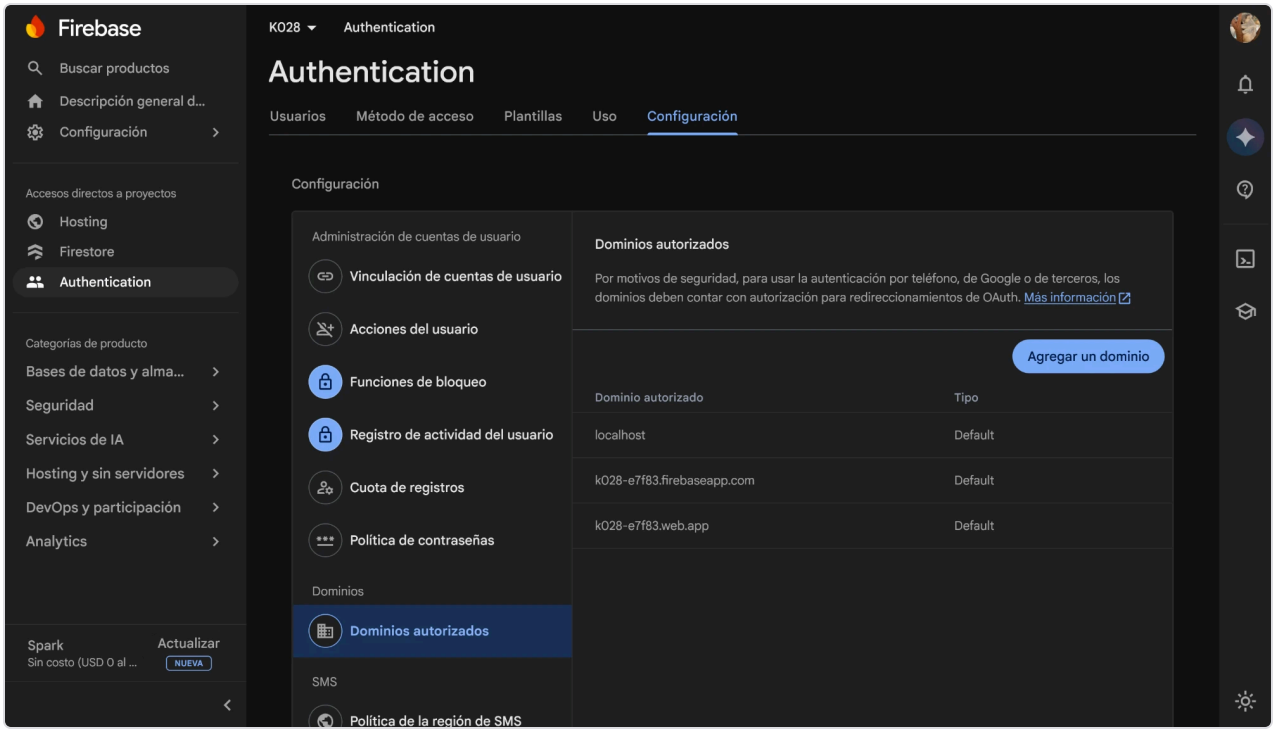


Figura 33. Dominios autorizados del proyecto.

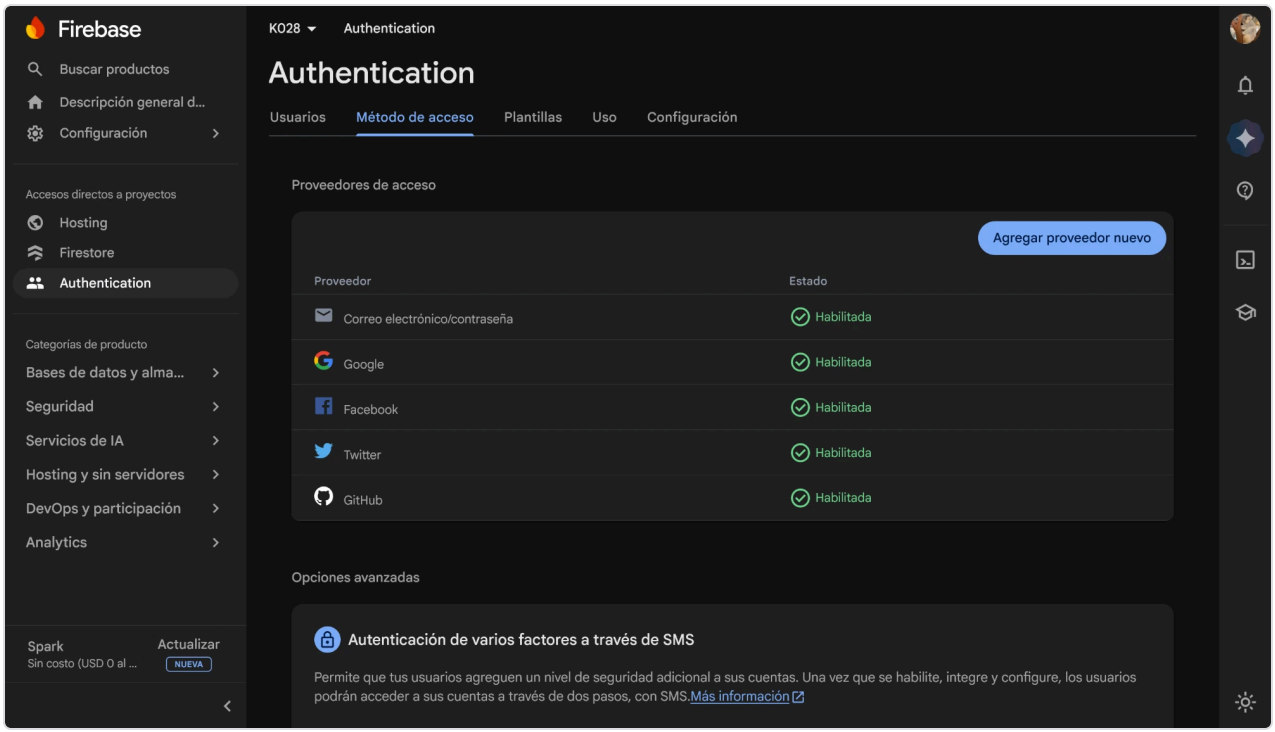


Figura 34. Proveedores de inicio de sesión habilitados.

> **Antes de la demostración.** Una app de Facebook en modo de desarrollo solo deja entrar a cuentas con algún rol en la app (administrador, desarrollador o probador). Para la demostración conviene agregar como probador a la persona que inicie sesión o usar proveedores que ya se hayan probado.

Lista de verificación posterior a la publicación

Comprobación	Resultado
☐ La URL pública abre con HTTPS y se ve la pantalla de bienvenida	
☐ Inicio de sesión con correo y contraseña	
☐ Inicio de sesión con Google	
☐ Inicio de sesión con al menos otro proveedor (Facebook, GitHub o X)	
☐ Crear un producto con 1 imagen y con 5 imágenes	
☐ Error claro al intentar subir 0 imágenes y más de 5	
☐ Editar y eliminar un producto (se eliminan también sus imágenes)	
☐ Cerrar sesión, entrar con otro usuario y no ver los productos del anterior	

Anexo. Guía para la demostración

Por qué esta solución es válida para un proyecto escolar y por qué en producción se usaría Cloud Storage

Firestore es una base de datos de documentos: no está pensada para guardar archivos y cada documento tiene un límite de 1 MiB. El servicio diseñado para archivos es Cloud Storage for Firebase, pero desde el 3 de febrero de 2026 exige el plan Blaze, que se vincula a una cuenta de facturación. Para que nadie necesite tarjeta, guardamos imágenes pequeñas en Base64 dentro de Firestore, que funciona en el plan gratuito.

	Firestore + Base64 (esta actividad)	Cloud Storage (producción)
Plan	Spark (sin costo, sin método de pago).	Blaze (pago por uso, con cuenta de facturación).
Tamaño	Cada imagen se reduce a 800 px y 300 KB como máximo para no acercarse al límite de 1 MiB por documento.	Pensado para archivos grandes: se pueden guardar las fotos originales.
Peso real	Base64 agrega aproximadamente un tercio al tamaño.	Se guarda el archivo binario tal cual.
Entrega	La imagen viaja dentro del documento y cada lectura de producto o imagen cuenta contra la cuota.	Cada archivo tiene su propia URL y se sirve de forma eficiente.
Escalabilidad	Válida para pocas imágenes pequeñas; no escala bien.	Escala a miles de archivos y usuarios.

Flujo de la demostración

La demostración es en vivo, con la URL pública de Hosting (no desde localhost), y participan los dos integrantes.

#	Qué se muestra	Responsable
1	Inicio de sesión con al menos dos proveedores.	
2	Creación de un producto con imágenes: el mínimo (1), el máximo (5) y el error al pasarse de 5.	
3	Edición y eliminación de un producto.	
4	Cierre de sesión y acceso con otro usuario, comprobando que no aparecen los productos del anterior.	
5	Consola de Firestore con los datos guardados.	

Conclusiones

Con este trabajo convertimos una aplicación que solo iniciaba sesión en un gestor de productos completo y publicado en internet. Lo más importante fue entender cómo se conectan los tres servicios: Authentication identifica a la persona, el uid se usa para armar la ruta `users/{uid}/products` y Firestore guarda ahí la información de cada usuario.

También aprendimos las limitaciones de las decisiones que tomamos. Guardar imágenes en Base64 dentro de Firestore es una solución práctica para un proyecto escolar, pero obliga a reducirlas y a cuidar el límite de 1 MiB; en un proyecto real se usaría Cloud Storage. Del mismo modo, el modo de prueba es cómodo para desarrollar, pero deja los datos abiertos, y en producción habría que protegerlos con reglas de seguridad y con validación también en el servidor.

Por último, publicar con Firebase Hosting nos mostró que terminar el código no es terminar el proyecto: hubo que instalar la CLI, desplegar, revisar los dominios autorizados de los proveedores y probar todo desde la URL pública. Dejar cada funcionalidad en su propio archivo y comentada nos facilitó repartir el trabajo y explicarlo en la demostración.